Networking, TLS & Retry Guide

Engineering Handbook · Internal · Owner: Platform Engineering

Purpose

This page defines how services on the platform (api-service, auth-service, worker-service, ingestion-service, gateway, scheduler) talk to each other and to external dependencies over the network. It covers transport security (TLS and mutual TLS), certificate lifecycle, and the retry, timeout, and connection-pool behaviour every outbound HTTP client must follow. The goal is to prevent a transient blip from becoming a self-inflicted outage and to keep service-to-service trust verifiable end to end.

Standards

TLS and mTLS trust

All service-to-service traffic is mutual TLS. Clients must trust the internal CA that signs peer certificates; do not disable verification. Mount the internal CA bundle and point the runtime at it, and ensure server certificates carry the correct SANs for the service DNS name.

```
import ssl, httpx
ctx = ssl.create_default_context(cafile="/etc/ssl/internal/ca-bundle.pem")
ctx.load_cert_chain(certfile="/etc/ssl/svc/tls.crt", keyfile="/etc/ssl/svc/tls.key")
# SSL_CERT_FILE=/etc/ssl/internal/ca-bundle.pem also works for libraries that read it
client = httpx.AsyncClient(verify=ctx, timeout=httpx.Timeout(3.0, connect=1.0))
```

- Rotate certificates well before expiry; automate issuance and renewal (ACME / cert-manager) rather than hand-rolling.
- Verify SANs match the DNS name the client dials; a name mismatch fails the handshake just like an untrusted CA.
- Never pin to a certificate that is not auto-renewed.

Timeouts, retries, and backoff

Every outbound client sets explicit connect and read timeouts — there is no such thing as an unbounded call. Retry only idempotent operations, cap the attempt count, and use exponential backoff with jitter so clients do not resynchronise into a stampede. Protect dependencies with a circuit breaker that opens after sustained failure.

```
import random, asyncio
async def call_with_retry(fn, attempts=3, base=0.2, cap=2.0):
   for i in range(attempts):
       try:
           return await fn()
       except (httpx.TransportError, httpx.HTTPStatusError):
           if i == attempts - 1:
               raise
```

```
            sleep = min(cap, base * 2 ** i) * (0.5 + random.random())  # full jitter
            await asyncio.sleep(sleep)
```

- Default budget: connect ≤ 1s, read ≤ 3s, ≤ 3 attempts. Tune per dependency SLO, never remove.
- Only retry GET / idempotent writes (those carrying an idempotency key). Never blind-retry non-idempotent POSTs.
- Respect DNS TTLs: dial service names, not cached IPs, and re-resolve on connection errors.
- Set the client keep-alive/idle timeout below the server or proxy idle timeout to avoid reset races.

Common failures and fixes

- Handshake fails with "certificate verify failed: unable to get local issuer certificate" → distribute the internal CA bundle to clients, mount the trust store / set SSL_CERT_FILE, and confirm SANs (BUG-1020).
- All calls to a dependency suddenly fail with "certificate has expired" → renew the cert, automate renewal, and alert well before expiry (BUG-1065).
- A brief downstream blip becomes a full outage as everyone retries at once → exponential backoff with jitter, capped attempts, and a circuit breaker (BUG-1067).
- Intermittent failures after a dependency IP change → respect DNS TTLs and re-resolve; stop caching resolved IPs in long-lived clients (BUG-1066).
- Requests hang and exhaust workers when a dependency is slow → set explicit connect and read timeouts on every client and fail fast (BUG-1068).
- Intermittent "connection reset by peer" (ECONNRESET) → set client idle timeout below the server's, recycle the pool, and retry idempotent calls on reset (BUG-1069).

Checklist

- mTLS enabled; internal CA bundle mounted and trusted by every client.
- Certificate expiry alerts fire with lead time; renewal is automated.
- Every outbound client sets connect and read timeouts.
- Retries use bounded attempts, exponential backoff, and full jitter.
- Retries restricted to idempotent operations; circuit breaker in place.
- Clients dial DNS names, honour TTLs, and recycle keep-alive connections.

Related incidents

BUG-1020, BUG-1065, BUG-1067, BUG-1066, BUG-1068, BUG-1069